

Guía del proyecto

Prácticas Preprofesionales: cuatro sprints sobre un producto heredado.

Carrera de Software, FISEI, UTA

Ciclo 2026-B

Modalidad A: equipos que trabajan en GitHub

Sprint 1: 16–24 sep 2026 · Sprints 2–4 por confirmar

Equipos de cinco (y cuatro: anexo B)

1 Antes de empezar

Este proyecto forma parte de un trabajo de titulación sobre cómo trabajan los equipos de desarrollo de la carrera. Durante cuatro sprints, seis equipos del paralelo trabajan sobre el mismo producto, desde la misma plantilla, con el mismo calendario.

Lo que el estudio registra es el historial de los dos repositorios de tu equipo y los resultados de su integración continua. Ninguna medición se asocia a tu nombre: los datos se guardan con identificadores anónimos, la tabla que los relaciona se custodia aparte y se destruye al terminar el estudio. Tu nota en la asignatura la pone el docente con los criterios de la sección 3; el estudio no califica a nadie.

Lo que esta guía te da

- Qué producto heredas y cómo levantarlo.
- Cómo trabaja tu equipo: ramas, pull requests, revisión.
- Las épicas y el orden en que se atacan.
- Qué historias tocan cada sprint y qué se entrega.
- El backlog completo, con criterios de aceptación.

Lo que no te da

- La solución. Las historias dicen qué debe quedar demostrable; cómo lo consigues lo decide tu equipo.
- Permiso para saltarte el orden. Las épicas se atacan en secuencia y las dependencias entre historias son reales.
- Un backlog que se termina. Ver la sección 7.

2 El producto que heredas

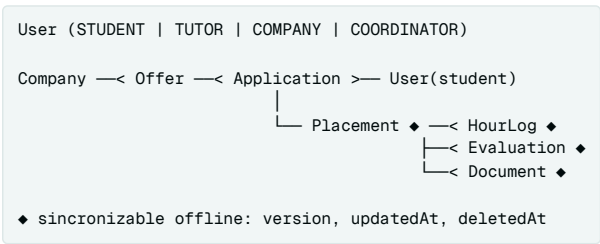
Prácticas Preprofesionales es una aplicación para que la unidad de vinculación de una carrera gestione las prácticas de sus estudiantes: ofertas de empresas con cupos, postulaciones, plazas, libro de horas validado por un tutor y acreditación por periodo.

Hoy ese ciclo se lleva en hojas de cálculo y correos. El estudiante registra sus horas en papel, muchas veces en una planta o en una comunidad rural donde no hay señal. Por eso el cliente funciona sin conexión: guarda las horas en el dispositivo y las sincroniza cuando vuelve la red. Esa capa es también donde el equipo anterior dejó los problemas más serios.

Cuatro roles

Rol	Qué hace en la aplicación
Estudiante	Postula a ofertas, registra horas de práctica (con o sin conexión), sube documentos de su plaza.
Tutor	Aprueba o rechaza las horas de las prácticas que tiene asignadas y evalúa al estudiante.
Empresa	Publica ofertas con cupos, decide postulaciones, evalúa al practicante.
Coordinación	Valida convenio y seguro, sigue la cohorte, acredita las prácticas al cierre del periodo.

Modelo de datos



Máquinas de estado

Entidad	Transiciones
Application	SUBMITTED → INTERVIEW → ACCEPTED REJECTED WITHDRAWN
Placement	PENDING_DOCS → ACTIVE → COMPLETED ABANDONED SUSPENDED
HourLog	DRAFT → SUBMITTED → APPROVED REJECTED

Reglas de negocio que el dominio sostiene

- Las postulaciones aceptadas de una oferta no superan sus cupos (`seats`).

- Un estudiante tiene como máximo una plaza en `ACTIVE`.
- Una plaza solo pasa a `ACTIVE` con convenio y seguro validados.
- Solo el tutor asignado aprueba las horas de su plaza.
- Una plaza debería pasar a `COMPLETED` cuando las horas aprobadas alcancen `requiredHours`. Es la regla prevista; todavía no hay código que la implemente.

Dos repositorios, uno por módulo

Módulo	Stack	Lo que importa saber
Backend API REST	NestJS, Prisma, PostgreSQL. JWT con cuatro roles. Endpoints <code>/sync/pull</code> y <code>/sync/push</code> para el cliente offline.	Cobertura de pruebas del 33 %. El módulo de evaluaciones no tiene ningún test.
Frontend SPA offline-first	React, Vite, Tailwind. Dexie sobre IndexedDB como copia local; service worker que precachea la app.	Cobertura del 16 %. La regla de oro: la interfaz nunca hace <code>fetch</code> de datos sincronizables, lee de Dexie.

Levantar el backend

```
pnpm install
cp .env.example .env
docker compose up -d
pnpm db:deploy
pnpm db:seed
pnpm dev      # http://localhost:3000/api
```

Levantar el frontend (con el backend arriba)

```
pnpm install
cp .env.example .env
pnpm dev      # http://localhost:5173
```

Requiere Node 24 y pnpm 10 o superior. Usuarios del seed, todos con contraseña `yura1234`: `coordinador@`, `tutor0@...` `tutor7@`, `empresa0@...` `empresa11@`, `estudiante0@...` `estudiante199@`, dominio `miyura.com`.

Lo que el equipo anterior dejó anotado

Cada repositorio trae un `KNOWN_ISSUES.md`: siete problemas en el backend (D-01 a D-07) y cuatro en el frontend (D-06, D-08 a D-10), ordenados por lo que más les dolió. Léelo completo antes de tocar nada. El archivo dice, con razón, que hay más problemas sin anotar: parte del trabajo es encontrarlos. Las épicas del backlog salen de esa lista y de lo que no está en ella.

3 Cómo trabaja tu equipo

Ramas y pull requests, igual para los dos repositorios

- `main` es la rama estable. Nadie hace push directo.
- `develop` es la rama de integración. **Todo pull request va contra `develop`.**
- Una rama por historia, nombrada con su clave: `feat/E1-02-cola-de-envio`, `fix/E3-04-jwt-secret`. Commits pequeños y frecuentes, con la clave de la historia en el mensaje.
- Un pull request por historia (dos si la historia toca los dos repositorios). La descripción dice qué cambia, cómo se prueba y qué queda fuera.
- Las pruebas van en el mismo PR que el código que cubren. Un PR que baja la cobertura del repositorio necesita una razón escrita.
- La integración continua del repositorio (`ci.yml`) corre en cada push: migra, typechequea, linta, prueba y construye. Un PR con la CI en rojo no se integra. **No desactives ni edites los workflows.**
- Al cierre de cada sprint, antes de la review, un PR de `develop` a `main` deja en `main` el estado del sprint.

Tu equipo trabaja en GitHub

1. **Repositorios: los crea el líder.** Dos, uno por módulo, en su cuenta de GitHub, desde las plantillas públicas:
 1. Abre `github.com/yura-edu/template-practicas-preprofesionales-backend-v1` → **Use this template** → **Create a new repository**.
 2. Owner: tu cuenta. Nombre: `practicas-preprofesionales-backend<equipo>` (por ejemplo `...-backend-a1`). Visibilidad **Public**. Marca **«Include all branches»**: sin eso el repositorio nace sin `develop`.
 3. Repite con `...-frontend-v1` → `practicas-preprofesionales-frontend<equipo>`.
 4. En cada repositorio, Settings → Collaborators → invita a cada integrante por su usuario de GitHub.
 5. Envía las dos URLs al docente el mismo día.

Los demás: acepta la invitación que te llega de GitHub y clona los dos repositorios. Comprueba que existan `main` y `develop`. Nunca desde un fork.

2. **Backlog.** El anexo A de esta guía es tu backlog: ocho épicas, 39 historias con sus criterios de aceptación. El plan de la sección 5 dice qué historias tocan cada semana. Lleva el seguimiento como el equipo prefiera: GitHub Issues, un tablero de Projects, una hoja compartida. Lo importante es que en la review se vea qué se cerró y qué no.
3. **Revisión entre pares.** Es opcional: el equipo decide si alguien lee el PR antes de integrarlo. Lo que no es opcional es que la CI esté en verde.
4. **Retroalimentación.** El docente revisa el trabajo del equipo al cierre de cada sprint, en la review, y ahí da la retroalimentación sobre lo entregado.

Criterio de evaluación del docente: correctitud funcional. Que la historia funcione como dicen sus criterios de aceptación, demostrada en la review sobre la aplicación corriendo.

4 Las ocho épicas, en orden de ataque

El backlog tiene 39 historias en ocho épicas, 178 puntos en total. El orden no es negociable: cada épica deja el terreno que la siguiente necesita. Un sprint trabaja sobre la épica en curso y no abre la siguiente mientras esa tenga historias pendientes.

1 E1 Confiabilidad de la sincronización offline

8 historias, 32 puntos

El estudiante registra horas en planta o en comunidad rural, muchas veces sin señal, y confía en que la app no las va a perder. Hoy esa confianza no está justificada.

El equipo anterior dejó tres síntomas anotados y varios sin anotar. El más citado es que **a veces se duplican horas y a veces desaparecen**, y solo pasa con red intermitente — justo las condiciones del sitio de práctica.

Esta épica existe para que el motor de sincronización deje de perder y de duplicar datos. Es la de mayor prioridad del proyecto: mientras no esté cerrada, cualquier otra funcionalidad que escriba datos hereda el mismo riesgo.

Cómo saber que terminó: un estudiante puede registrar diez bloques de horas sin conexión, cerrar la app, recuperar señal, y encontrar exactamente diez registros en el servidor — ni nueve ni once.

2 E3 Seguridad y control de acceso

6 historias, 21 puntos

El sistema maneja datos personales de estudiantes, evaluaciones que afectan su titulación, y actas de acreditación. Hoy tiene agujeros que un estudiante curioso encuentra en una tarde.

Esta épica no añade funcionalidad: hace que la que existe sea confiable. Trátala como bloqueante para cualquier despliegue fuera del entorno de pruebas.

Cómo saber que terminó: un usuario autenticado no puede leer ni modificar nada que no le corresponda, y una sesión no dura para siempre.

3 E2 Integridad de cupos y rendimiento del catálogo

4 historias, 13 puntos

Una oferta de prácticas tiene cupos finitos, y hoy nada garantiza que se respeten cuando dos personas deciden a la vez. Además, las pantallas que más usa la empresa se degradan con el volumen real de un semestre.

Esta épica cierra la brecha entre "el sistema dice que hay 3 cupos" y "el sistema garantiza que solo entran 3".

Cómo saber que terminó: dos aceptaciones simultáneas sobre la última plaza dejan exactamente un aceptado, y la lista de postulaciones de una oferta con 200 candidatos abre sin espera perceptible.

4 E6 Calidad del libro de horas

6 historias, 30 puntos

El libro de horas es la pantalla más usada del sistema y la que peor está por dentro: la misma validación escrita en tres sitios con tres límites distintos, un servicio que hace de todo, un formulario de 350 líneas, y una lista que se arrastra cuando crece.

Nada de esto se ve desde fuera hasta que hay que cambiar algo. Entonces se ve todo junto.

Cómo saber que terminó: cambiar el límite de horas por jornada se hace en un sitio, y el libro con 500 registros se desplaza sin saltos.

5 E5 Acta de acreditación

4 historias, 21 puntos

El acta es el producto final del sistema: el documento con el que la coordinación certifica que un estudiante cumplió sus prácticas. Hoy existe como una pantalla, pero la función que decide quién acredita es la que más miedo da del proyecto, y no hay forma de sacar el acta del navegador.

Cómo saber que terminó: la coordinación puede cerrar un periodo, descargar el acta firmada en PDF, y el equipo puede cambiar las reglas de acreditación sin sudar.

6 E4 Evaluación por rúbrica configurable

5 historias, 29 puntos

Hoy la evaluación son tres criterios fijos escritos en el código: técnico, comunicación y puntualidad, del 1 al 5. Cada carrera evalúa distinto, y cambiar los criterios hoy exige tocar backend, frontend y base de datos a la vez.

Esta épica convierte la evaluación en algo que la coordinación configura, sin dejar inconsistentes las evaluaciones ya registradas.

Cómo saber que terminó: la coordinación puede crear una rúbrica de cinco criterios con pesos distintos, usarla en un periodo, y las evaluaciones del periodo anterior siguen leyéndose bien.

7 E7 Panel de seguimiento para coordinación

3 historias, 16 puntos

La coordinación hoy solo puede mirar el acta, que llega tarde por definición. Necesita ver cómo avanza la cohorte mientras el periodo está abierto, cuando todavía se puede hacer algo.

Cómo saber que terminó: la coordinación entra un miércoles cualquiera y sabe qué prácticas van retrasadas y qué tutores tienen horas sin revisar.

8 E8 Periodos académicos

3 historias, 16 puntos

El periodo hoy es una cadena de texto que se deriva de la fecha de inicio de cada práctica. Funciona hasta que dos carreras tienen calendarios distintos, o hasta que alguien necesita cerrar un periodo y que deje de aceptar registros.

Cómo saber que terminó: la coordinación abre y cierra periodos, y un periodo cerrado no admite registros nuevos.

5 Semana a semana

Cuatro sprints. El Sprint 1 va del miércoles 16 sep al jueves 24 sep: 9 días, con fechas confirmadas. Los tres siguientes duran una semana y sus fechas se confirman antes de que arranque cada uno. Cada sprint cierra con una review el último día, donde el equipo demuestra lo hecho sobre la aplicación corriendo.

Semana 0 8-15 sep Pretest, sorteo de equipos, repositorios, levantar el stack.	Sprint 1 16-24 sep E1 cierra. 32 pts.	Sprint 2 fechas por confirmar E3 cierra; E2 avanza. ~24 pts, provisional.	Sprint 3 fechas por confirmar E2 cierra; E6 avanza. ~25 pts, provisional.	Sprint 4 fechas por confirmar E6 cierra; E5 avanza. ~25 pts, provisional.	Cierre tras el Sprint 4 Demo final, postest, entrega de cierre.
---	--	--	--	--	--

Semana 0: 8-15 sep

- Pretest individual de competencia técnica (45 minutos, en clase). Con su resultado se forman los equipos por sorteo.
- El equipo queda fijo hasta el cierre del Sprint 4: no hay cambios de integrantes durante el periodo.
- El líder crea los dos repositorios desde las plantillas e invita al resto (sección 3); los demás aceptan la invitación.
- Clona los dos repositorios, levanta el stack, entra con los usuarios del seed y recorre la aplicación con cada rol.
- Lee el [README.md](#) y el [KNOWN_ISSUES.md](#) de cada repositorio. Corre las pruebas y mira qué cubren.
- Lee las épicas (sección 4) y las historias del Sprint 1 (anexo A). Reparte quién arranca con qué.

Antes del miércoles 16 sep no se integra nada en `develop` ni en `main`. El Sprint 1 arranca con el repositorio tal como salió de la plantilla; explora, anota y prepara, pero no subas cambios.

El plan para un equipo de cinco

Capacidad: 5 puntos por integrante y semana, así que 32 puntos en el Sprint 1 (9 días) y 25 por sprint de una semana. El reparto sigue el orden de ataque y no abre una épica nueva mientras la actual tenga historias pendientes; por eso algún sprint cierra con uno o dos puntos libres. Solo el Sprint 1 es un compromiso: los demás son el reparto previsto si cada sprint dura una semana.

Sprint 1

16-24 sep · 9 días · review jueves 24 sep

E1 cierra un estudiante puede registrar diez bloques de horas sin conexión, cerrar la app, recuperar señal, y encontrar exactamente diez registros en el servidor — ni nueve ni once.

Clave	Historia	Pts	Tipo	Depende de
E1-01	Reproducir la pérdida de horas cuando la red falla a mitad del envío	3	spike	—
E1-02	No borrar la cola de envío hasta que el servidor confirme	5	bug	E1-01
E1-03	Evitar que un reintento cree la hora dos veces	5	bug	—
E1-04	Rechazar la edición offline de una hora que el tutor ya resolvió	5	feature	—
E1-05	Dejar de saltarse registros que comparten el mismo instante	3	bug	—
E1-06	Reintentar los envíos fallidos con espera creciente	5	feature	—
E1-07	Que el indicador nunca diga "sincronizado" con pendientes	3	bug	—
E1-08	Mantener el estado de sincronización coherente entre pestañas	3	bug	—

Compromiso: 8 historias, 32 de 32 puntos.

Cierre: PR de develop a main antes de la review. Seguimiento del equipo al día antes de la review.

Sprint 2

fechas por confirmar · review al cierre

Plan provisional. Supone un sprint de una semana (25 puntos). Las fechas y las historias definitivas las confirma el docente antes de que arranque el sprint.

E3 cierra un usuario autenticado no puede leer ni modificar nada que no le corresponda, y una sesión no dura para siempre.

E2 avanza sin cerrar: 1 de 4 historias de «Integridad de cupos y rendimiento del catálogo» (1 acumuladas al cierre).

Clave	Historia	Pts	Tipo	Depende de
E3-01	Auditar quién puede hacer qué en cada endpoint	5	spike	–
E3-02	Impedir que un tutor apruebe horas de una práctica ajena	3	bug	E3-01
E3-03	Hacer que la sesión caduque	5	feature	–
E3-04	Exigir el secreto de firma en vez de usar uno por defecto	2	bug	–
E3-07	Impedir que una empresa decida postulaciones de ofertas ajenas	3	bug	E3-01
E3-06	Revisar que ninguna respuesta filtre datos sensibles	3	chore	–
E2-01	Demostrar el sobrecupo con dos aceptaciones simultáneas	3	spike	–

Previsto: 7 historias, 24 de 25 puntos.

Cierre: PR de develop a main antes de la review. Seguimiento del equipo al día antes de la review.

Sprint 3

fechas por confirmar · review al cierre

Plan provisional. Supone un sprint de una semana (25 puntos). Las fechas y las historias definitivas las confirma el docente antes de que arranque el sprint.

E2 cierra dos aceptaciones simultáneas sobre la última plaza dejan exactamente un aceptado, y la lista de postulaciones de una oferta con 200 candidatos abre sin espera perceptible.

E6 avanza sin cerrar: 3 de 6 historias de «Calidad del libro de horas» (3 acumuladas al cierre).

Clave	Historia	Pts	Tipo	Depende de
E2-02	Garantizar que nunca se acepten más postulaciones que cupos	5	bug	E2-01
E2-03	Abrir la lista de postulaciones sin una consulta por candidato	3	bug	–
E2-04	Acelerar el libro de horas con el índice que falta	2	chore	–
E6-01	Unificar la validación de horas que hoy vive en tres sitios	5	chore	–
E6-02	Guardar las fechas de la práctica sin desfase de zona horaria	5	bug	–
E6-03	Separar responsabilidades del servicio de horas	5	chore	E6-01

Previsto: 6 historias, 25 de 25 puntos. **Cierre:** PR de `develop` a `main` antes de la review. Seguimiento del equipo al día antes de la review.

Sprint 4

fechas por confirmar · review al cierre

Plan provisional. Supone un sprint de una semana (25 puntos). Las fechas y las historias definitivas las confirma el docente antes de que arranque el sprint.

E6 cierra cambiar el límite de horas por jornada se hace en un sitio, y el libro con 500 registros se desplaza sin saltos.

E5 avanza sin cerrar: 2 de 4 historias de «Acta de acreditación» (2 acumuladas al cierre).

Clave	Historia	Pts	Tipo	Depende de
E6-04	Separar el formulario de registro de horas	5	chore	E6-01
E6-05	Que el libro de horas no se arrastre con muchos registros	5	bug	–
E6-06	Probar los hooks que leen de la base local	5	chore	–
E5-01	Fijar con tests las reglas de acreditación que hay hoy	5	chore	–
E5-02	Reescribir la decisión de acreditación de forma legible	5	chore	E5-01

Previsto: 5 historias, 25 de 25 puntos. **Cierre:** PR de `develop` a `main` antes de la review. Seguimiento del equipo al día antes de la review.

Si tu equipo es de cuatro

Los equipos de cuatro integrantes tienen 25 puntos en el Sprint 1 (9 días) y 20 por sprint de una semana. El reparto es el mismo, en el mismo orden, y se corta antes: en cuatro sprints llegan a cerrar E1, E3, E2. El anexo B trae el plan exacto para cuatro. El docente te confirma qué plan sigue tu equipo.

6 Entregables

Cada sprint, el día de la review

Entregable	Qué se revisa
Historias integradas en <code>develop</code>	Cada historia comprometida entra por uno o más pull requests, con sus pruebas. Una historia sin PR integrado no está hecha.
<code>main</code> actualizada	Antes de la review, un PR de <code>develop</code> a <code>main</code> deja en <code>main</code> el estado del sprint.
Demo en la review	El equipo muestra cada historia funcionando sobre la aplicación corriendo, contra sus criterios de aceptación. Lo que no se puede enseñar funcionando no cuenta como entregado.
Estado del trabajo	Con el seguimiento del equipo al día: qué se cerró, qué se arrastra, qué se descubrió.

Al cierre del Sprint 4

Entregable	Qué se revisa
<code>main</code> con todo lo integrado	Las épicas cerradas se demuestran de punta a punta con el criterio «cómo saber que terminó» de la sección 4.
Demo de cierre	Recorrido completo del producto por rol, con lo que cambió respecto a la plantilla.
<code>KNOWN_ISSUES.md</code> actualizado	Lo que se arregló sale de la lista; lo que se descubrió y no se arregló entra, con el mismo formato y la misma honestidad que el original.
Nota de cierre	Un archivo <code>CIERRE.md</code> en el backend, de una página: qué cerraron, qué quedó en el backlog y por qué, y qué le dirían al siguiente equipo.

7 No vas a terminar el backlog, y está bien

El backlog suma 178 puntos. Un equipo de cinco tiene 107 en los cuatro sprints; uno de cuatro, 85. Desde el primer día, 13 historias (o 18, si son cuatro) quedan fuera del plan. No es un fallo del plan: es el plan.

Un backlog que cabe justo no obliga a nadie a priorizar, y priorizar es la mitad del ejercicio. Ningún producto real tiene el backlog vacío; lo que distingue a un buen equipo es qué elige cerrar y qué deja dicho que no va a hacer.

- **Lo que cuenta es lo demostrable.** Tres historias cerradas de verdad, con pruebas y funcionando en la review, valen más que seis a medias.
- **Si terminas antes,** toma la siguiente historia de la misma épica, en orden. No abras una épica nueva mientras la actual tenga historias pendientes, y no saltes el orden de ataque.
- **Si no llegas,** dílo en la review. Una historia que se arrastra al siguiente sprint desplaza a otra; el equipo decide cuál y lo deja escrito.
- **Los spikes no son opcionales.** Reproducir el fallo con un test antes de arreglarlo es lo que separa arreglarlo de creer que lo arreglaste. Por eso el fix depende del spike y no al revés.
- **Lo que queda en el backlog no se pierde.** Sigue visible, sigue ordenado, y es lo primero que va a leer el equipo que herede el proyecto.

Anexo A Backlog completo

Las 39 historias, por épica y en orden de ejecución. Cada una trae la historia de usuario, el contexto, los criterios de aceptación, notas técnicas y la definición de terminado. Es la misma información que gobierna el plan de la sección 5.

E1 Confiabilidad de la sincronización offline 8 historias, 32 puntos, épica 1 de 8

E1-01 Reproducir la pérdida de horas cuando la red falla a mitad del envío

3 pts spike prioridad urgente Offline Calidad

Historia

Como **equipo de desarrollo**, quiero un test que reproduzca de forma fiable la pérdida de horas al fallar la red, para poder arreglarla sabiendo que quedó arreglada.

Contexto

`KNOWN_ISSUES.md` menciona duplicados al reintentar, pero hay reportes de horas que directamente desaparecen. Antes de tocar nada hay que poder provocarlo a voluntad: un arreglo sin test que lo demuestre es una conjetura.

Criterios de aceptación

- Existe un test en el repo de frontend que simula un fallo de red durante `pushOutbox` y **falla hoy**, demostrando que las operaciones se pierden.
- El test no depende de tiempos ni de `sleep`: el fallo se inyecta mockeando la capa de red.
- El reporte del spike documenta, con la evidencia del test, **en qué orden ocurren las operaciones** que causan la pérdida.
- Queda registrado si el problema afecta también a documentos y evaluaciones, o solo a horas.

Notas técnicas

Mira `src/offline/sync/push.ts`. Presta atención al **orden** entre vaciar la cola local y confirmar la respuesta del servidor.

Definición de terminado

El test está en la rama, falla por la razón correcta, y el equipo entero entiende el mecanismo.

E1-02 No borrar la cola de envío hasta que el servidor confirme

5 pts bug prioridad urgente Offline Deuda técnica Frontend depende de E1-01

Historia

Como **estudiante en práctica**, quiero que mis horas registradas sin conexión sigan ahí aunque la red se corte en mitad del envío, para no tener que acordarme de cuáles se perdieron.

Contexto

Depende de **E1-01**, que deja el test que demuestra el fallo.

Criterios de aceptación

- Dado que hay operaciones en la cola local, cuando el envío al servidor falla por red, entonces las operaciones **siguen en la cola** y se reintentan en la siguiente sincronización.
- Dado que el envío responde correctamente, cuando se aplican los resultados, entonces solo se eliminan de la cola las operaciones que el servidor confirmó.
- Una operación que el servidor rechaza no se pierde en silencio: queda marcada y visible para el usuario con el motivo que dio el servidor.
- El test de **E1-01** pasa.
- No se introducen duplicados: el test de duplicación existente sigue en verde.

Notas técnicas

Cuidado al reordenar: la cola es también lo único que sabe qué identificador local corresponde a cada operación enviada. Si la vacías tarde, asegúrate de que la reconciliación de identificadores sigue funcionando.

Definición de terminado

Verificado a mano apagando el backend a mitad de una sincronización, con las horas intactas después.

E1-03 Evitar que un reintento cree la hora dos veces

5 pts bug prioridad urgente Offline Deuda técnica Backend

Historia

Como **estudiante en práctica**, quiero que reintentar un envío no me cree la misma hora dos veces, para no tener que pedirle a coordinación que borre duplicados.

Contexto

Es el síntoma que `KNOWN_ISSUES.md` describe como *"a veces se duplican horas al reintentar, no supimos por qué"*. El cliente ya manda un identificador único por operación y el servidor lo guarda; falta cerrar el círculo.

Criterios de aceptación

- Dado que el cliente envía una operación con un identificador que el servidor ya procesó, cuando llega el reintento, entonces el servidor **no crea un registro nuevo** y devuelve el resultado original.
- La respuesta del reintento es indistinguible de la primera desde el punto de vista del cliente.
- Existe un test de integración que envía la misma operación dos veces y verifica que solo hay una fila en base de datos.
- La solución funciona con envíos concurrentes de la misma operación, no solo secuenciales.

Notas técnicas

Revisa la tabla `sync_operations` en el repo de backend: ya existe y se escribe, pero fíjate en **cuándo** se consulta.

Definición de terminado

El test de duplicación pasa, y el escenario manual de "enviar dos veces el mismo identificador" deja una sola fila.

E1-04 Rechazar la edición offline de una hora que el tutor ya resolvió

5 pts feature prioridad alta Offline Deuda técnica Backend

Historia

Como **tutor académico**, quiero que mi aprobación o mi rechazo no se pierdan porque el estudiante tenía una edición vieja en el teléfono, para no tener que revisar dos veces lo mismo.

Contexto

El README documenta que el servidor es la autoridad sobre el estado: si una hora ya está aprobada o rechazada, la edición offline debería rechazarse y mostrársele al estudiante. **Compara esa documentación con lo que hace el código** — no coinciden.

Criterios de aceptación

- Dado que una hora está en estado aprobado o rechazado, cuando llega una edición del cliente, entonces el servidor la rechaza y devuelve el motivo.
- Dado que ambos lados están en borrador o enviada, cuando llega la edición, entonces se aplica y gana la más reciente.
- El cliente muestra al estudiante las ediciones rechazadas con una explicación entendible, no un código de error.
- El estudiante puede ver el estado que quedó en el servidor sin recargar la aplicación.
- Existe un test por cada una de las dos ramas.

Notas técnicas

El cliente ya envía la versión que tenía cuando editó. Búscala en el cuerpo del envío y fíjate si el servidor la usa para algo.

Definición de terminado

El comportamiento coincide con lo que el README ya promete, y el README sigue siendo cierto.

E1-05 Dejar de saltarse registros que comparten el mismo instante

3 pts bug prioridad alta Offline Backend

Historia

Como **estudiante**, quiero que la descarga de cambios no se salte registros, para que mi libro de horas en el teléfono coincida con el del servidor.

Contexto

La descarga avanza con un cursor. Si dos filas se actualizan en el mismo milisegundo y el cursor solo mira la marca de tiempo, la segunda se pierde para siempre — el cursor ya pasó de largo.

Es difícil de ver a mano y fácil de provocar con una carga masiva.

Criterios de aceptación

- Existe un test que crea dos registros con la misma marca de tiempo exacta y verifica que **ambos** llegan al cliente.
- El cursor desempata de forma estable, de modo que dos descargas seguidas nunca devuelven la misma fila dos veces ni omiten ninguna.
- El cursor sigue siendo opaco para el cliente: el frontend no interpreta su contenido.
- La descarga por páginas sigue funcionando cuando hay más filas que el límite.

Notas técnicas

Mira cómo se construye el filtro de la consulta en el módulo de sincronización del backend, y compáralo con lo que el cursor realmente transporta.

Definición de terminado

El test de mismo milisegundo pasa, y una descarga completa de los 4.000 registros del seed no pierde ninguno.

E1-06 Reintentar los envíos fallidos con espera creciente

5 pts

feature

prioridad media

Offline

Frontend

Historia

Como **estudiante con señal intermitente**, quiero que la app reintente sola mis envíos sin agotarme la batería, para no tener que estar pendiente de pulsar "sincronizar".

Contexto

Hoy la sincronización se dispara al arrancar, al recuperar la conexión y cada minuto. Cuando falla, no hay reintento propio: se espera al siguiente ciclo. Con señal que va y viene, eso desperdicia intentos y batería.

Los campos para contar intentos y guardar el último error ya existen en la cola local — **nadie los lee ni los escribe**.

Criterios de aceptación

- Un envío fallido se reintentará con esperas crecientes (por ejemplo 1s, 2s, 4s, 8s), con un tope configurable.
- El contador de intentos y el último error quedan registrados en la cola local.
- Tras agotar los reintentos, la operación queda marcada como fallida y visible para el usuario, no se descarta.
- Los reintentos se cancelan si el dispositivo pierde la conexión, y se reanudan al recuperarla.
- No se solapan dos ciclos de sincronización a la vez.

Definición de terminado

Verificado cortando y restaurando la red varias veces seguidas sin que se pierda ni se duplique nada.

E1-07 Que el indicador nunca diga "sincronizado" con pendientes

3 pts

bug

prioridad alta

Offline

Deuda técnica

Frontend

Historia

Como **estudiante sin conexión**, quiero que el indicador de sincronización nunca me diga "sincronizado" mientras queden registros en cola, para saber si puedo cerrar la app tranquilo.

Contexto

Es el **D-08** de [KNOWN_ISSUES.md](#). Al terminar un push, `runSync` hace dos llamadas seguidas: primero `setStatus({ syncing: false, lastSyncAt })` y **una línea después** `setStatus({ pending: await db.outbox.count() })`. Entre las dos hay una vuelta al event loop en la que el estado dice "ya terminé" con un contador de pendientes viejo.

Mirándolo en pantalla no lo vas a ver: son milisegundos. Pero un test que observe la secuencia de estados del indicador lo atrapa siempre — y cualquier componente que reaccione al cambio de `syncing` lee el `pending` equivocado.

Ojo: el contador **sí** se actualiza al encolar (`enqueue()` ya lo hace). El problema está solo en el cierre del ciclo de sync.

Criterios de aceptación

- Existe un test que suscribe al estado, corre un ciclo de sincronización y **falla hoy** al detectar un estado intermedio con `syncing: false` y `pending` sin recalcular.
- Tras el cambio, en ninguna transición del estado coinciden `syncing: false` y un `pending` que no corresponda al contenido real de la cola.
- El test anterior pasa, y sigue pasando el test existente de `status.ts`.
- `lastSyncAt` se sigue registrando solo cuando el push terminó sin error.

Notas técnicas

Dos formas válidas: un único `setStatus` con `syncing`, `lastSyncAt` y `pending` juntos, o calcular `pending` **antes** de apagar `syncing`. Explica en el PR cuál elegiste y por qué. Mira `scheduler.ts` y `status.ts`.

Definición de terminado

El test de secuencia está en la rama y el componente `SyncIndicator` no tiene forma de pintar "sincronizado" con pendientes.

E1-08 Mantener el estado de sincronización coherente entre pestañas

3 pts bug prioridad media Offline Frontend

Historia

Como **estudiante que abre la app en dos pestañas**, quiero ver el mismo estado en las dos, para no dudar de si mis horas se enviaron.

Contexto

Las dos pestañas comparten la base local del navegador pero **no** el estado de sincronización en memoria. Cada una lleva su propio contador y se pisan.

Es el tipo de problema que solo aparece cuando alguien deja una pestaña abierta en el laboratorio y abre otra.

Criterios de aceptación

- Dos pestañas de la misma sesión muestran el mismo número de pendientes y la misma hora de último sync.
- Sincronizar en una pestaña actualiza el indicador de la otra sin recargar.
- No se disparan dos sincronizaciones simultáneas desde pestañas distintas sobre la misma cola.

Notas técnicas

`BroadcastChannel` o el evento `storage` sirven para esto. Elige uno y explica en el PR por qué.

Definición de terminado

Probado con dos pestañas abiertas registrando y sincronizando de forma alterna.

E3 Seguridad y control de acceso 6 historias, 21 puntos, épica 2 de 8

E3-01 Auditar quién puede hacer qué en cada endpoint

5 pts spike prioridad urgente Seguridad Backend Calidad

Historia

Como **equipo de desarrollo**, quiero un inventario de qué rol puede llamar a cada endpoint y sobre qué datos, para saber cuántos agujeros tenemos antes de taparlos de uno en uno.

Contexto

El sistema tiene cuatro roles. Los guards verifican el **rol**, pero rol y pertenencia no son lo mismo: ser tutor no significa ser el tutor *de esa* práctica.

Criterios de aceptación

- Existe una tabla en el repositorio con una fila por endpoint: método, ruta, roles permitidos, y si comprueba **pertenencia** además de rol.
- Cada fila marcada como "no comprueba pertenencia" incluye una prueba concreta —un `curl`— que demuestra si es explotable o no.
- Los hallazgos explotables quedan creados como tareas hijas de esta épica, priorizadas.
- El inventario cubre los dos repositorios.

Notas técnicas

Empieza por los endpoints que reciben un identificador en la ruta: son los candidatos naturales.

Definición de terminado

El equipo sabe exactamente cuántos agujeros hay y cuáles duelen.

E3-02 Impedir que un tutor apruebe horas de una práctica ajena

3 pts bug prioridad urgente Seguridad Backend Deuda técnica depende de E3-01

Historia

Como **coordinación**, quiero que solo el tutor asignado pueda aprobar las horas de su practicante, para que el acta de acreditación signifique algo.

Contexto

Sale del inventario de **E3-01**. Compruébalo tú antes de arreglarlo: entra como un tutor, busca una hora pendiente de una práctica que **no** tutorizas, y apruébala.

Criterios de aceptación

- Dado un tutor que no es el asignado a la práctica, cuando intenta aprobar o rechazar una hora, entonces recibe un 403.
- El tutor asignado sigue pudiendo aprobar y rechazar con normalidad.
- La coordinación conserva los permisos que ya tenía.
- Existe un test para el caso denegado y otro para el permitido.
- El mismo criterio se aplica a cualquier otro endpoint del inventario con el mismo patrón.

Definición de terminado

El `curl` que antes devolvía 200 ahora devuelve 403, y está en un test.

E3-03 Hacer que la sesión caduque

5 pts feature prioridad alta Seguridad Backend Frontend

Historia

Como **estudiante que usa un computador del laboratorio**, quiero que mi sesión caduque, para que quien lo use después no entre con mi cuenta.

Contexto

El token que emite el sistema **no tiene fecha de expiración**. Decodifícalo y compruébalo. Un token filtrado sirve para siempre, y cerrar sesión en una pestaña no afecta a la otra.

Criterios de aceptación

- El token de acceso incluye expiración, con una vida corta y configurable.
- Existe una forma de renovar la sesión sin volver a pedir credenciales, y renovarla **invalida** el token anterior.
- Al expirar, el cliente lleva al usuario al inicio de sesión con un mensaje claro, sin pantallas rotas.
- Cerrar sesión en una pestaña cierra la sesión en las demás.
- Cerrar sesión limpia también los datos locales del navegador: el siguiente usuario del equipo no ve nada del anterior.
- Existen tests de token expirado y de renovación.

Notas técnicas

Cuidado con el orden: si el cliente no maneja el 401 antes de que expiren los tokens, el usuario verá errores raros en vez de la pantalla de inicio de sesión.

Definición de terminado

Probado en dos navegadores y con el reloj adelantado.

E3-04 Exigir el secreto de firma en vez de usar uno por defecto

2 pts bug prioridad alta Seguridad Backend Deuda técnica

Historia

Como **responsable del despliegue**, quiero que la aplicación se niegue a arrancar sin su secreto de firma, para que nunca llegue a producción firmando con uno conocido.

Contexto

`KNOWN_ISSUES.md` lo admite: *"lo pusimos para no pelear con el entorno local y se quedó"*. El secreto por defecto está en el código, es público, y cualquiera puede firmarse un token de coordinación con él.

Criterios de aceptación

- Si falta la variable de entorno del secreto, la aplicación **no arranca** y el mensaje dice exactamente qué falta.
- No queda ningún valor por defecto en el código.
- El archivo de ejemplo de entorno documenta la variable.
- El README explica cómo generar un secreto adecuado.
- La configuración de CI define la variable para que la suite siga en verde.

Definición de terminado

Arrancar sin la variable falla con un mensaje útil; con ella, todo funciona igual que antes.

E3-07 Impedir que una empresa decida postulaciones de ofertas ajenas

3 pts bug prioridad urgente Seguridad Backend depende de E3-01

Historia

Como **empresa**, quiero que solo yo pueda ver y decidir las postulaciones a mis ofertas, para que otra empresa no acepte ni rechace a mis candidatos.

Contexto

Sale del inventario de **E3-01**. `GET /offers/:offerId/applications` y `PATCH /applications/:id/decide` están detrás de `@Roles(COMPANY, COORDINATOR)`, pero el servicio nunca comprueba que la oferta pertenezca a la empresa de quien llama. Compruébalo tú: entra como una empresa, toma el id de una postulación a una oferta que **no** es tuya, y decídela.

Es el mismo patrón que **E3-02**, en otro módulo: el guard mira el rol, nadie mira la pertenencia.

Criterios de aceptación

- Dado un usuario `COMPANY` cuya `companyId` no es la de la oferta, cuando lista sus postulaciones o decide una de ellas, entonces recibe un 403.
- La empresa dueña de la oferta sigue listando y decidiendo con normalidad.

- La coordinación conserva el acceso que ya tenía a ambos endpoints.
- Existe un test para el caso denegado y otro para el permitido en cada endpoint.
- La comprobación vive en el servicio, no solo en el controlador: llamar al servicio directo con una empresa ajena también falla.

Notas técnicas

El usuario ya trae `companyId` y la oferta también. Fíjate en cómo `assertPlacementAccess` resuelve la pertenencia para horas y copia el patrón, no el código.

Definición de terminado

El `curl` de una empresa ajena que antes devolvía 200 ahora devuelve 403, y está en un test.

E3-06 Revisar que ninguna respuesta filtre datos sensibles

3 pts chore prioridad alta Seguridad Backend Calidad

Historia

Como **estudiante**, quiero que mis datos personales no viajen en respuestas donde no hacen falta, para que un fallo en otro sitio no los exponga.

Contexto

Cuando una consulta devuelve un usuario completo, devuelve **todas** sus columnas. Revisa qué sale realmente por la API en las pantallas que muestran personas.

Criterios de aceptación

- Existe una revisión escrita de todos los endpoints que devuelven usuarios, indicando qué campos exponen.
- Ninguna respuesta incluye credenciales ni campos internos.
- Cada consulta que devuelve usuarios acota explícitamente los campos.
- Se añade un test que falla si una respuesta vuelve a incluir un campo sensible.

Notas técnicas

Un test que compruebe la forma de la respuesta es más duradero que la revisión manual: la revisión caduca al siguiente PR.

Definición de terminado

La revisión está en el repositorio y el test protege contra la regresión.

E2 Integridad de cupos y rendimiento del catálogo 4 historias, 13 puntos, épica 3 de 8

E2-01 Demostrar el sobrecupo con dos aceptaciones simultáneas

3 pts spike prioridad alta Backend Calidad

Historia

Como **equipo de desarrollo**, quiero una prueba que provoque el sobrecupo de forma repetible, para arreglarlo con evidencia y no de oído.

Contexto

La verificación de cupos lee cuántas postulaciones hay aceptadas y compara contra el total. Entre esa lectura y la escritura hay una ventana.

Criterios de aceptación

- Existe una prueba que lanza dos aceptaciones concurrentes sobre una oferta con **una sola** plaza libre y demuestra que ambas pasan.
- La prueba es determinista: no depende de suerte ni de repetirla muchas veces.
- El reporte explica dónde está la ventana exactamente, citando archivo y línea.

Definición de terminado

El equipo puede provocar el sobrecupo a voluntad y sabe por qué ocurre.

E2-02 Garantizar que nunca se acepten más postulaciones que cupos

5 pts bug prioridad alta Backend Deuda técnica depende de E2-01

Historia

Como **empresa**, quiero que el sistema me impida aceptar más practicantes de los que ofrecí, para no tener que desdecirme con un estudiante.

Contexto

Depende de **E2-01**.

Criterios de aceptación

- Dado una oferta con la última plaza libre, cuando dos aceptaciones llegan a la vez, entonces exactamente una tiene éxito y la otra recibe un error claro.

- El error que ve la empresa explica que la oferta se llenó, no un fallo genérico.
- La garantía no depende solo del código de aplicación: la base de datos también la sostiene.
- La prueba de **E2-01** ahora demuestra que solo una pasa.
- Aceptar sigue funcionando normalmente cuando hay cupos de sobra.

Notas técnicas

Hay varias formas: una transacción con bloqueo de fila, o una restricción en base de datos que haga imposible el estado inválido. Discutid cuál preferís y dejadlo escrito en el PR — la segunda es más difícil de saltarse por accidente.

Definición de terminado

El sobrecupo es imposible incluso llamando a la API directamente, sin pasar por la interfaz.

E2-03 Abrir la lista de postulaciones sin una consulta por candidato

3 pts bug prioridad media Backend Rendimiento Deuda técnica

Historia

Como **empresa con una oferta popular**, quiero ver mis postulaciones sin esperar, para poder revisarlas en una sentada.

Contexto

`KNOWN_ISSUES.md` lo describe como *"con menos de 200 postulaciones no se nota; arriba de eso la pantalla tarda varios segundos"*, y admite que nunca lo midieron.

Empieza midiendo. Ojo con el dato: el seed de la plantilla crea 200 postulaciones pero las reparte entre 36 ofertas (~5 por oferta), así que primero tienes que cargar 200 sobre **una misma oferta** — un script sobre el seed o un bucle de `POST /applications` sirven.

Criterios de aceptación

- Antes de tocar código, queda registrado en el PR cuántas consultas SQL dispara la pantalla hoy con 200 postulaciones, y cuánto tarda.
- Después del cambio, el número de consultas **no crece con el número de postulaciones**.
- La respuesta con 200 postulaciones baja de 500 ms en local.
- La forma de la respuesta de la API no cambia: el cliente no necesita ajustes.

Notas técnicas

Activa el log de consultas del ORM para contarlas. La pista está en cómo se resuelve el estudiante de cada postulación.

Definición de terminado

El PR incluye los números de antes y después.

E2-04 Acelerar el libro de horas con el índice que falta

2 pts chore prioridad media Backend Rendimiento Deuda técnica

Historia

Como **estudiante con un semestre de horas registradas**, quiero que mi libro abra rápido, para revisarlo sin frustrarme.

Contexto

La consulta que lista las horas de una práctica recorre la tabla entera. Con los 4.000 registros del entorno de pruebas ya se nota; con varios semestres acumulados, más.

Criterios de aceptación

- Queda registrado en el PR el plan de ejecución **antes** del cambio, mostrando el recorrido secuencial.
- Se añade el índice mediante una migración, no a mano en la base.
- El plan de ejecución después del cambio usa el índice.
- La mejora de tiempo queda medida y anotada en el PR.

Notas técnicas

`EXPLAIN ANALYZE` sobre la consulta que filtra horas por práctica y ordena por fecha te dice qué falta.

Definición de terminado

La migración está aplicada y el plan de ejecución lo confirma.

E6-01 Unificar la validación de horas que hoy vive en tres sitios

5 pts chore prioridad alta Calidad Deuda técnica Backend Frontend

Historia

Como **equipo de desarrollo**, quiero una sola definición de qué es un registro de horas válido, para que cambiarla no exija recordar tres lugares.

Contexto

[KNOWN_ISSUES.md](#) lo admite y además admite que ya divergieron: *"nadie recuerda cuál es la correcta"*. Empezar por averiguar cuántas copias hay realmente y qué límite tiene cada una — la nota puede quedarse corta.

Criterios de aceptación

- Queda documentado en el PR cuántas copias había, dónde, y qué límite aplicaba cada una.
- La regla queda definida en **un solo lugar** por repositorio, y ambos repos coinciden en el valor.
- El límite es configurable, no un número escrito en medio de una función.
- El usuario recibe el mismo mensaje de error valide quien valide.
- El detector de duplicación deja de señalar estas copias.
- Existen tests de los límites, incluidos los bordes.

Notas técnicas

La coordinación tiene que poder decir cuál es el límite correcto. Si nadie lo sabe, pregúntalo antes de elegir uno.

Definición de terminado

Cambiar el límite es una sola edición, y hay un test que lo demuestra.

E6-02 Guardar las fechas de la práctica sin desfase de zona horaria

5 pts bug prioridad alta Backend Deuda técnica

Historia

Como **estudiante**, quiero que la fecha que registro sea la que queda guardada, para que mis horas no aparezcan en el día equivocado.

Contexto

La fecha del registro llega desde el dispositivo y se guarda tal cual, sin normalizar. Un teléfono con la hora mal, o simplemente una diferencia horaria, y el registro cae en otro día.

Tampoco se comprueba que la fecha esté dentro del periodo de la práctica: hoy se pueden registrar horas de antes de empezar.

Criterios de aceptación

- La fecha se normaliza en el servidor a la zona horaria de la institución, configurable.
- Una fecha fuera del rango de la práctica se rechaza con un mensaje que dice cuál es el rango válido.
- No se pueden registrar horas con fecha futura.
- Existen tests con dispositivos en zonas horarias distintas que producen el mismo resultado.
- Los registros existentes no cambian de día al desplegar: si hace falta migrarlos, la migración es explícita y reversible.

Notas técnicas

Decide y documenta si la fecha es "día calendario" o "instante". No es lo mismo y mezclarlo es lo que produce estos errores.

Definición de terminado

Un dispositivo en otra zona horaria registra el mismo día que uno local.

E6-03 Separar responsabilidades del servicio de horas

5 pts chore prioridad media Calidad Backend Deuda técnica depende de E6-01

Historia

Como **equipo de desarrollo**, quiero poder cambiar cómo se notifica al tutor sin leer la lógica de persistencia, para que los cambios sean pequeños y revisables.

Contexto

[KNOWN_ISSUES.md](#) : *"empezó siendo el CRUD de horas y terminó haciendo validación, persistencia, notificación al tutor y agregación de reportes. Cada vez que hay que tocar algo ahí, hay que leerlo entero."*

Criterios de aceptación

- Las responsabilidades quedan en unidades separadas, cada una con un propósito que se puede nombrar en una frase.

- Ningún archivo resultante pasa de 150 líneas.
- Los tests existentes siguen pasando sin modificarse.
- La API pública no cambia: los controladores no necesitan ajustes.
- La cobertura del área no baja.

Notas técnicas

Empieza extrayendo lo que menos dependencias tiene. Cada extracción, un commit: un PR de 350 líneas movidas de sitio es imposible de revisar.

Definición de terminado

Un revisor puede entender cada archivo nuevo sin abrir los demás.

E6-04 Separar el formulario de registro de horas

5 pts chore prioridad media Calidad Frontend Deuda técnica depende de E6-01

Historia

Como **equipo de desarrollo**, quiero poder probar la lógica del formulario de horas sin montar la interfaz, para detectar errores de validación con un test rápido.

Contexto

`KNOWN_ISSUES.md` : el formulario mezcla estado, validación, formateo de fechas, escritura en la base local y encolado, todo en un archivo.

Depende de **E6-01**, que unifica la validación: hacerlo al revés significa mover tres copias en vez de una.

Criterios de aceptación

- La lógica de estado y validación queda fuera del componente visual y es testeable sin renderizar.
- El componente visual se ocupa de presentar y de avisar de eventos, nada más.
- Ningún archivo resultante pasa de 150 líneas.
- Existen tests de la lógica extraída: validación, transiciones de estado, construcción de la operación a encolar.
- El comportamiento visible no cambia, incluido el registro sin conexión.

Definición de terminado

Verificado registrando horas sin conexión, igual que antes.

E6-05 Que el libro de horas no se arrastre con muchos registros

5 pts bug prioridad media Rendimiento Frontend

Historia

Como **estudiante al final del semestre**, quiero desplazarme por mi libro de horas sin saltos, para revisarlo cómodamente en el teléfono.

Contexto

Con pocos registros no se nota. Con un semestre entero, sí — y es justo cuando el estudiante más lo revisa, al cerrar.

Criterios de aceptación

- Antes de tocar nada, queda medido en el PR el tiempo de renderizado con 500 registros.
- Tras el cambio, desplazarse por 500 registros va fluido en un teléfono de gama media.
- Los totales no se recalculan en cada renderizado.
- El comportamiento no cambia: se ven los mismos datos, con el mismo canalón de sincronización.
- La mejora queda medida y anotada en el PR.

Notas técnicas

Mide antes de optimizar. Puede que el problema no esté donde parece: quizá sea el cálculo de totales y no la lista.

Definición de terminado

El PR incluye la medición de antes y después en un dispositivo real o emulado.

E6-06 Probar los hooks que leen de la base local

5 pts chore prioridad media Calidad Frontend Deuda técnica

Historia

Como **equipo de desarrollo**, quiero tests de los hooks que leen la base local, para cambiar el esquema sin romper pantallas en silencio.

Contexto

`KNOWN_ISSUES.md` : *"no supimos testear hooks que tocan IndexedDB y lo dejamos."*

La buena noticia es que la infraestructura ya está: el proyecto trae una implementación de IndexedDB para pruebas y hay tests del esquema local que la usan. El obstáculo real era menor de lo que parecía.

Crterios de aceptacin

- Los hooks que leen la base local tienen tests que ejercitan lectura real, no mocks del hook.
- Los tests cubren: estado de carga inicial, datos presentes, y coleccin vaca.
- La cobertura de esa carpeta pasa de 0% a ms del 70%.
- Los tests no dependen del orden en que se ejecutan.

Notas tcnicas

Mira el test del esquema local que ya existe: usa el mismo patrn de limpiar y reabrir entre casos.

Definicin de terminado

La carpeta deja de aparecer en cero en el informe de cobertura.

E5 Acta de acreditacin 4 historias, 21 puntos, epica 5 de 8

E5-01 Fijar con tests las reglas de acreditacin que hay hoy

5 pts chore prioridad alta Calidad Backend Deuda tcnica

Historia

Como **equipo de desarrollo**, quiero tests que cubran todos los caminos de la decisin de acreditacin, para poder reescribirla sin cambiar resultados por accidente.

Contexto

`KNOWN_ISSUES.md` dice que la funcin "da miedo tocarla": el linter la marca por complejidad y solo tiene dos tests, as que cualquier cambio es a ciegas. Y decide si alguien se titula a tiempo.

Estos son **tests de caracterizacin**: describen lo que el cdigo hace hoy, no lo que debera hacer. Si encuentras una regla que te parece mal, crala como tarea aparte; aqu solo se congela el comportamiento.

Crterios de aceptacin

- Cada rama de decisin tiene al menos un test que la ejercita.
- Los tests cubren los cuatro niveles de resultado posibles.
- La cobertura del archivo supera el 90%.
- Ningn test cambia el comportamiento: la suite pasa sin tocar la funcin.
- Las reglas quedan documentadas en una tabla, en el propio PR: entrada → resultado esperado.

Notas tcnicas

El informe del linter te dice cuantos caminos hay. Usalo como lista de verificacin.

Definicin de terminado

La tabla de reglas est escrita y todos los caminos tienen test.

E5-02 Reescribir la decisin de acreditacin de forma legible

5 pts chore prioridad media Calidad Backend Deuda tcnica depende de E5-01

Historia

Como **equipo de desarrollo**, quiero poder leer las reglas de acreditacin de un vistazo, para cambiarlas cuando la normativa cambie sin arriesgar el resultado de nadie.

Contexto

Depende de **E5-01**: sin la red de tests, esta historia es imprudente.

Son condiciones anidadas hasta seis niveles. Cada nueva regla las hace peor.

Crterios de aceptacin

- Todos los tests de **E5-01** siguen pasando **sin modificarlos**. Ese es el criterio central: si tienes que tocar un test, cambiaste el comportamiento.
- El linter ya no marca la funcin por complejidad.
- Las reglas se leen como reglas: alguien de coordinacin debera poder revisarlas y entenderlas.
- Aadir una regla nueva no obliga a tocar las existentes.
- El PR explica la estructura elegida y por qu.

Notas tcnicas

Hay varias formas: reglas como datos, clausulas de guarda tempranas, una tabla de decisin. Cualquiera vale si las pruebas no cambian.

Definicin de terminado

Complejidad bajo el umbral, tests intactos, y una persona ajena entiende las reglas leyndolas.

E5-03 Descargar el acta del periodo en PDF

8 pts feature prioridad media Backend Frontend

Historia

Como **coordinación**, quiero descargar el acta del periodo en PDF, para archivarla y entregarla a secretaría académica.

Contexto

Hoy el acta solo existe en pantalla. El trámite real necesita un documento.

Criterios de aceptación

- La coordinación puede descargar el acta de un periodo en PDF desde la pantalla de acreditación.
- El PDF incluye: periodo, fecha de emisión, y por estudiante su nombre, empresa, horas aprobadas sobre requeridas, nivel de acreditación y observaciones.
- Los estudiantes salen ordenados alfabéticamente por apellido.
- El documento lleva un pie con la fecha de generación y quién lo generó.
- Un periodo sin prácticas genera un PDF válido que dice que no hay registros, no un archivo corrupto.
- Con 200 estudiantes el PDF se genera en menos de 10 segundos y pagina correctamente.

Notas técnicas

Decidid si el PDF se arma en el servidor o en el cliente y justificadlo en el PR. Si es en el servidor, cuidado con bloquear el proceso durante la generación.

Definición de terminado

Descargado y abierto con los 200 estudiantes del entorno de pruebas.

E5-04 Avisar a la coordinación de las prácticas atascadas

3 pts feature prioridad baja Frontend

Historia

Como **coordinación**, quiero ver de un vistazo qué prácticas no van a acreditar y por qué, para intervenir antes de que se acabe el periodo.

Contexto

El acta ya calcula las razones por las que alguien no acredita, pero solo se ven al final, cuando ya no hay margen.

Criterios de aceptación

- La pantalla de acreditación permite filtrar por nivel de resultado.
- Cada práctica en riesgo muestra la razón concreta: faltan horas, falta convenio, falta evaluación del tutor.
- Se muestra cuántos días quedan del periodo.
- Los filtros se reflejan en la URL, para poder compartir la vista.
- La pantalla sigue el sistema de diseño y funciona a 360px.

Definición de terminado

La coordinación puede responder "¿quién no va a acreditar y por qué?" en menos de un minuto.

E4 Evaluación por rúbrica configurable 5 historias, 29 puntos, épica 6 de 8

E4-01 Cubrir con tests el módulo de evaluaciones antes de tocarlo

5 pts chore prioridad alta Calidad Backend Deuda técnica

Historia

Como **equipo de desarrollo**, quiero tests que fijen el comportamiento actual de las evaluaciones, para poder cambiarlo sin romperlo.

Contexto

`KNOWN_ISSUES.md` lo dice sin rodeos: *"zero tests. Lo hicimos en la última semana y no alcanzamos."* Es el módulo con menos cobertura del proyecto, y el que esta épica va a reescribir entero.

Escribir los tests **primero** no es ceremonia: es lo que permite distinguir "lo cambié" de "lo rompi".

Criterios de aceptación

- Existen tests del envío de evaluación: caso válido, práctica inexistente, y validación de rangos.
- Existen tests del listado por práctica.
- Los tests describen el comportamiento **actual**, aunque alguno parezca incorrecto; si encuentras algo que parece un fallo, créalo como tarea aparte en vez de arreglarlo aquí.
- La cobertura del módulo pasa del 0% a más del 70%.

Definición de terminado

El informe de cobertura muestra el módulo cubierto, y la suite pasa.

E4-02 Modelar rúbricas con criterios y pesos

8 pts feature prioridad alta Backend depende de E4-01

Historia

Como **coordinación de carrera**, quiero definir los criterios con los que se evalúa una práctica y cuánto pesa cada uno, para que la evaluación refleje lo que mi carrera valora.

Contexto

Depende de **E4-01**.

Los datos de evaluación hoy se guardan como una estructura libre con tres campos fijos. Hay que darles forma sin invalidar lo ya registrado.

Criterios de aceptación

- Existe un modelo de rúbrica con nombre, estado y una lista ordenada de criterios.
- Cada criterio tiene etiqueta, descripción, escala y peso.
- Los pesos de una rúbrica suman 100, y el sistema lo impide si no es así.
- Una rúbrica en uso no se puede modificar: se versiona.
- La migración deja las evaluaciones existentes legibles, asociadas a una rúbrica inicial que reproduce los tres criterios actuales.
- Existen tests del modelo y de la validación de pesos.

Notas técnicas

Piensa qué pasa con una evaluación ya registrada cuando su rúbrica cambia. La respuesta que elijas condiciona el resto de la épica: déjala escrita en el PR.

Definición de terminado

La migración corre sobre la base con datos y ninguna evaluación previa se pierde.

E4-03 Administrar rúbricas desde la interfaz de coordinación

8 pts feature prioridad media Frontend depende de E4-02

Historia

Como **coordinación**, quiero crear y editar rúbricas sin pedirle nada a desarrollo, para ajustar la evaluación entre periodos.

Contexto

Depende de **E4-02**.

Criterios de aceptación

- La coordinación puede crear una rúbrica, añadir criterios, reordenarlos y asignar pesos.
- La interfaz impide guardar si los pesos no suman 100, y explica cuánto falta o sobra.
- Se puede duplicar una rúbrica existente como punto de partida.
- Una rúbrica en uso se muestra como no editable, con la razón.
- La pantalla sigue el sistema de diseño del proyecto: superficie de registro, no tarjetas.
- Funciona a 360px de ancho.

Definición de terminado

Una persona de coordinación puede crear una rúbrica de cinco criterios sin ayuda.

E4-04 Evaluar usando la rúbrica del periodo

5 pts feature prioridad media Frontend Backend depende de E4-02, E4-03

Historia

Como **tutor académico**, quiero que el formulario de evaluación muestre los criterios que definió mi carrera, para evaluar lo que corresponde.

Contexto

Depende de **E4-02** y **E4-03**.

Criterios de aceptación

- El formulario se construye a partir de la rúbrica vigente, no de una lista fija en el código.
- Cada criterio muestra su descripción y su escala.
- Al guardar, la evaluación queda ligada a la **versión** de rúbrica con la que se llenó.
- Una evaluación registrada con una rúbrica anterior se sigue viendo con sus criterios originales.
- El cálculo de la nota respeta los pesos.

Definición de terminado

Una evaluación con la rúbrica antigua y otra con la nueva conviven y se leen correctamente.

E4-05 Reflejar los pesos de la rúbrica en el acta de acreditación

3 pts feature prioridad media Backend depende de E4-04

Historia

Como **coordinación**, quiero que el acta use la nota ponderada de la rúbrica, para que el resultado sea coherente con lo que definí.

Contexto

Depende de **E4-04**. El cálculo de acreditación hoy promedia tres números a partes iguales.

Criterios de aceptación

- El acta usa la nota ponderada según los pesos de la rúbrica con la que se evaluó.
- Las prácticas evaluadas con rúbricas distintas se calculan cada una con la suya.
- El umbral mínimo para acreditar es configurable, no un número escrito en el código.
- Existen tests del cálculo con al menos dos rúbricas distintas.

Definición de terminado

El acta de un periodo con dos rúbricas distintas da resultados correctos para ambas.

E7 Panel de seguimiento para coordinación 3 historias, 16 puntos, épica 7 de 8

E7-01 Ver el avance de la cohorte en un panel

8 pts feature prioridad media Frontend Backend

Historia

Como **coordinación**, quiero ver el avance de todas las prácticas del periodo en una pantalla, para detectar problemas mientras hay tiempo de resolverlos.

Criterios de aceptación

- El panel muestra, para el periodo activo: prácticas activas, completadas y abandonadas; promedio de avance de horas; y cuántas horas esperan revisión del tutor.
- Se puede filtrar por empresa y por tutor.
- Cada indicador enlaza al listado que lo compone: un número que no se puede abrir no sirve para actuar.
- La consulta responde en menos de un segundo con 200 prácticas.
- Sigue el sistema de diseño y funciona a 360px.

Notas técnicas

Cuidado con resolver esto con muchas consultas pequeñas. Mide cuántas dispara la pantalla antes de darla por buena.

Definición de terminado

Verificado contra el periodo del entorno de pruebas con sus 200 prácticas.

E7-02 Avisar de las prácticas sin actividad reciente

5 pts feature prioridad baja Backend

Historia

Como **coordinación**, quiero enterarme de las prácticas que llevan semanas sin registros, para llamar al estudiante antes de que sea tarde.

Criterios de aceptación

- El sistema identifica prácticas activas sin registros de horas nuevos en un plazo configurable.
- El aviso incluye estudiante, empresa, tutor y días sin actividad.
- Los avisos se pueden marcar como atendidos y no reaparecen mientras siga sin haber actividad.
- Existen tests del cálculo, incluido el caso de una práctica que arrancó hace poco y aún no tiene registros.

Definición de terminado

Con datos de prueba, el listado identifica correctamente las prácticas inactivas.

E7-03 Exportar el seguimiento a CSV

3 pts feature prioridad baja Backend Frontend depende de E7-01

Historia

Como **coordinación**, quiero exportar el seguimiento a CSV, para cruzarlo con los sistemas de la universidad.

Criterios de aceptación

- La exportación respeta los filtros aplicados en pantalla.
- El archivo incluye cabeceras en español y se abre correctamente en Excel, con tildes y ñ.

- Las fechas van en un formato inequívoco.
- Un conjunto vacío genera un CSV con solo cabeceras, no un archivo roto.

Notas técnicas

La codificación es donde falla siempre: pruébalo abriendo el archivo en Excel, no solo en un editor de texto.

Definición de terminado

Abierto en Excel con tildes correctas.

E8 Periodos académicos 3 historias, 16 puntos, épica 8 de 8

E8-01 Convertir el periodo académico en una entidad

8 pts feature prioridad media Backend

Historia

Como **coordinación**, quiero que el periodo académico sea algo que administro, para que sus fechas y su estado no dependan de cómo se derivan de otro dato.

Contexto

Hoy el periodo se calcula a partir de la fecha de inicio de la práctica, con una regla escrita en el código.

Es frágil: revisa qué pasa con una práctica que empieza el 1 de enero.

Criterios de aceptación

- Existe un modelo de periodo con código, fechas de inicio y fin, y estado.
- Cada práctica se asocia a un periodo, en vez de derivarlo de sus fechas.
- La migración asigna periodo a las prácticas existentes sin perder datos y **corrige** las que hoy están mal clasificadas.
- El acta filtra por el periodo asociado.
- Existen tests de la asignación, incluidos los bordes de año.

Notas técnicas

Antes de migrar, cuenta cuántas prácticas existentes cambian de periodo con la corrección. Si son muchas, avisa a coordinación: puede haber actas ya emitidas con el dato viejo.

Definición de terminado

La migración corre sobre la base con datos y el conteo de correcciones queda documentado en el PR.

E8-02 Cerrar un periodo y bloquear registros nuevos

5 pts feature prioridad media Backend Frontend depende de E8-01

Historia

Como **coordinación**, quiero cerrar un periodo, para que nadie registre ni modifique horas después de emitir el acta.

Contexto

Depende de **E8-01**.

Criterios de aceptación

- La coordinación puede cerrar un periodo, con confirmación explícita.
- Cerrado, no se aceptan registros ni ediciones de horas de ese periodo, ni desde la interfaz ni desde la API.
- El error que ve el estudiante explica que el periodo está cerrado.
- Un periodo cerrado se puede reabrir, y la reapertura queda registrada con quién y cuándo.
- Las operaciones que llegan sin conexión y pertenecen a un periodo cerrado se rechazan con un motivo claro, no se pierden en silencio.
- Existen tests de registro sobre periodo cerrado y de reapertura.

Notas técnicas

El último criterio es el que se olvida: alguien puede tener horas en la cola local desde antes del cierre. Comprueba qué pasa hoy en ese caso.

Definición de terminado

Probado registrando sin conexión, cerrando el periodo, y sincronizando después.

E8-03 Consultar el histórico de periodos anteriores

3 pts feature prioridad baja Frontend depende de E8-02

Historia

Como **coordinación**, quiero consultar las prácticas de periodos anteriores, para responder consultas de titulación sin pedirle nada a desarrollo.

Criterios de aceptación

- Un selector permite cambiar de periodo en las pantallas de acreditación y seguimiento.
- Los periodos cerrados se muestran en modo lectura, sin acciones que los modifiquen.
- El periodo seleccionado se refleja en la URL, para poder compartir la vista.
- Un periodo sin prácticas muestra un estado vacío que lo explica.

Definición de terminado

La coordinación puede consultar cualquier periodo pasado sin ayuda.

Anexo B Plan para equipos de cuatro

Capacidad de 25 puntos en el Sprint 1 (9 días) y 20 por sprint de una semana: 85 en total.
Mismo orden de ataque, mismas reglas de reparto. Como en la sección 5, solo el Sprint 1 es compromiso.

Sprint 1: 16–24 sep

Cierra: —. Avanza: E1. 24 de 25 puntos.

Clave	Historia	Pts	Tipo
E1-01	Reproducir la pérdida de horas cuando la red falla a mitad del envío	3	spike
E1-02	No borrar la cola de envío hasta que el servidor confirme	5	bug
E1-03	Evitar que un reintento cree la hora dos veces	5	bug
E1-04	Rechazar la edición offline de una hora que el tutor ya resolvió	5	feature
E1-05	Dejar de saltarse registros que comparten el mismo instante	3	bug
E1-07	Que el indicador nunca diga "sincronizado" con pendientes	3	bug

Sprint 2: fechas por confirmar

Cierra: E1. Avanza: E3. 18 de 20 puntos. Plan provisional: supone una semana.

Clave	Historia	Pts	Tipo
E1-06	Reintentar los envíos fallidos con espera creciente	5	feature
E1-08	Mantener el estado de sincronización coherente entre pestañas	3	bug
E3-01	Auditar quién puede hacer qué en cada endpoint	5	spike
E3-02	Impedir que un tutor apruebe horas de una práctica ajena	3	bug
E3-04	Exigir el secreto de firma en vez de usar uno por defecto	2	bug

Sprint 3: fechas por confirmar

Cierra: E3. Avanza: E2. 19 de 20 puntos. Plan provisional: supone una semana.

Clave	Historia	Pts	Tipo
E3-03	Hacer que la sesión caduque	5	feature
E3-07	Impedir que una empresa decida postulaciones de ofertas ajenas	3	bug
E3-06	Revisar que ninguna respuesta filtre datos sensibles	3	chore
E2-01	Demostrar el sobrecupo con dos aceptaciones simultáneas	3	spike
E2-02	Garantizar que nunca se acepten más postulaciones que cupos	5	bug

Sprint 4: fechas por confirmar

Cierra: E2. Avanza: E6. 20 de 20 puntos. Plan provisional: supone una semana.

Clave	Historia	Pts	Tipo
E2-03	Abrir la lista de postulaciones sin una consulta por candidato	3	bug
E2-04	Acelerar el libro de horas con el índice que falta	2	chore
E6-01	Unificar la validación de horas que hoy vive en tres sitios	5	chore
E6-02	Guardar las fechas de la práctica sin desfase de zona horaria	5	bug
E6-03	Separar responsabilidades del servicio de horas	5	chore

Quedan en el backlog, sin sprint: E6-04, E6-05, E6-06, E5-01, E5-02, E5-03, E5-04, E4-01, E4-02, E4-03, E4-04, E4-05, E7-01, E7-02, E7-03, E8-01, E8-02, E8-03.